



TIPS • 2026

10 Practical Tips for **UI/UX & AI** Engineers

Battle-tested lessons from shipping 11+ production projects — combining design intuition with AI engineering. Real tactics, not theory.

UI/UX Design

AI Engineering

Full-Stack

MLOps

Febri Rizki • UI/UX Designer & AI Engineer

Swipe →

// UI/UX DESIGN • PART 1

Design **Systems** Are Not Optional

01 Build Design Tokens First

Before drawing a single screen, define colors, typography scale, spacing, and shadows as tokens. Figma Variables + Tailwind config in sync = consistency on autopilot.

→ Action: Create `tokens.json` before wireframes

02 Auto Layout > Manual Padding

Learn Figma Auto Layout deeply. Components built with Auto Layout respond to content changes — no more "design broke when copy changed" reviews.

→ Action: Refactor 1 old component to Auto Layout today

03 Accessibility = Free Market Expansion

WCAG 2.1 AA isn't just compliance — it's 15% more users who can actually use your product. Contrast ratios, focus states, semantic HTML.

→ Action: Run `axe DevTools` on your latest project

Test With **5 Users** Not 50

5

users reveal 85% of usability problems.
Source: Nielsen Norman Group.
Don't wait for "perfect" — test early, iterate fast.

04

Prototype Before You Code

Figma's interactive prototype + 5 user tests = weeks saved in development. A clickable prototype catches flow issues that wireframes can't.

→ Action: Prototype the next feature before sprint planning

05

Micro-interactions = Delight

Hover states, loading animations, success toasts — these aren't "nice to have". They're the difference between a tool people use and one they love.

→ Action: Add 3 micro-interactions to current sprint

Prompt Engineering Is Real Engineering

06 Separate System & User Prompts

System prompt = persona & rules. User prompt = the actual query. Mixing them = unpredictable output. Keeping them separate = consistent, testable behavior.

```
// Bad
messages: [{ role: 'user', content: 'You are a tutor. Explain...' }]

// Good
messages: [
  { role: 'system', content: tutorSystemPrompt },
  { role: 'user', content: userQuery }
]
```

07 Cache Aggressively • Save 70% on Costs

In TRADIX, I implemented two-layer caching (memory + persistent). Result: 70% fewer API calls, 60% faster response, 80% lower cost. Same model, smarter retrieval.

→ Action: Add Redis or in-memory cache to your next LLM call

RAG > Fine-tuning (90% of the time)

08 Use RAG for Domain Knowledge

Fine-tuning is expensive, slow, and hard to update. RAG (Retrieval-Augmented Generation) lets you swap docs in seconds. For 90% of use cases, RAG wins.

Fine-tuning

\$\$\$ · Hours to retrain · Hard to debug ·
Hallucination risk on updates

RAG

Free · Seconds to update · Debuggable
· Source-attributable

09 Run ML Inference on Background Thread

On mobile (Flutter/React Native), never block the UI thread for ML. Use Isolate (Flutter) or Web Worker (Web). Users won't tolerate jank for AI features.

```
// Flutter: TFLite on Isolate
Future<Prediction> predict(File image) async {
  return await compute(_runInference, image.path);
  // UI stays 60fps during inference
}
```

// FULL-STACK · MLOPS

Automate **ML Retraining** From Day One

10 CI/CD for ML = MLflow + GitHub Actions

Don't manually retrain. Set up a weekly cron job that runs `mlflow run MLProject --env-manager=local` via GitHub Actions. Models stay fresh, drift drops 50%.

```
# .github/workflows/retrain.yml
on:
  schedule: [{ cron: '0 0 * * 0' }] # weekly
jobs:
  retrain:
    runs-on: ubuntu-latest
    steps:
      - run: mlflow run MLProject --env-manager=local
```

→ Action: Set up MLflow Project spec this week

// BONUS MINDSET

Ship > Perfect

The best design, the cleanest code, the smartest AI — none of it matters if you don't ship. Real users teach you more than 100 hours of planning.

11+

projects shipped.

Each one taught me more than the last 10 books I read.

Bias toward deployment.

The Hybrid Advantage

If you can design AND code AND integrate AI, you're 10x more valuable than a specialist. Don't pick one path — blend them. That's where the magic happens.

// RECAP

The 10 Tips Quick Recap

UI/UX · 01

Design Tokens first

UI/UX · 02

Auto Layout > Manual

UI/UX · 03

WCAG 2.1 from day 1

UI/UX · 04

Prototype before code

UI/UX · 05

Micro-interactions matter

AI · 06

System ≠ User prompt

AI · 07

Cache aggressively

AI · 08

RAG > Fine-tuning

AI · 09

ML on background thread

MLOps · 10

Automate retraining



Which Tip Will You Apply?

Pick ONE tip. Apply it to your project this week. Then come back and tell me how it went — I read every comment.

Febri Rizki

UI/UX Designer & AI Engineer

➤ [linkedin.com/in/febririzki95](https://www.linkedin.com/in/febririzki95)

➤ github.com/febririzki95

↑ Swipe back · Save this carousel · Share with a teammate